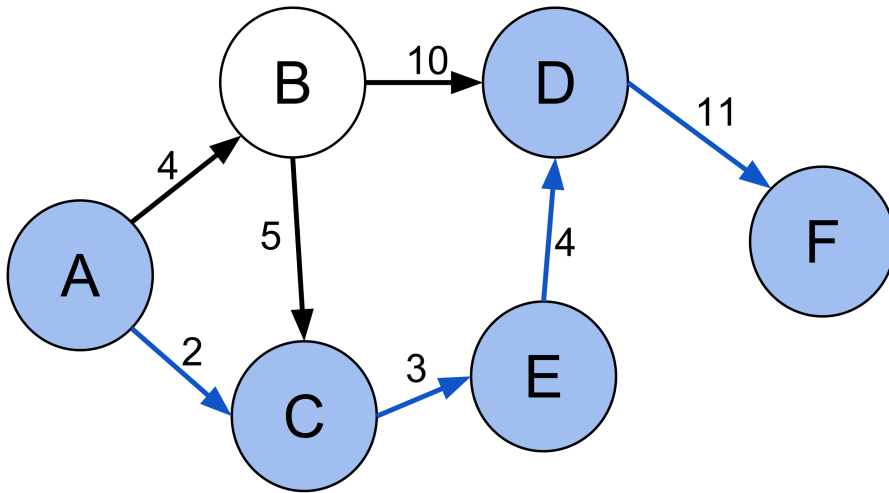


Shortest Path (เส้นทางที่สั้นที่สุด)

การหาค่าเส้นทางที่มีน้ำหนักรวมต่ำสุดระหว่างโหนดสองโหนดในกราฟ



อัลกอริทึมที่นิยม:

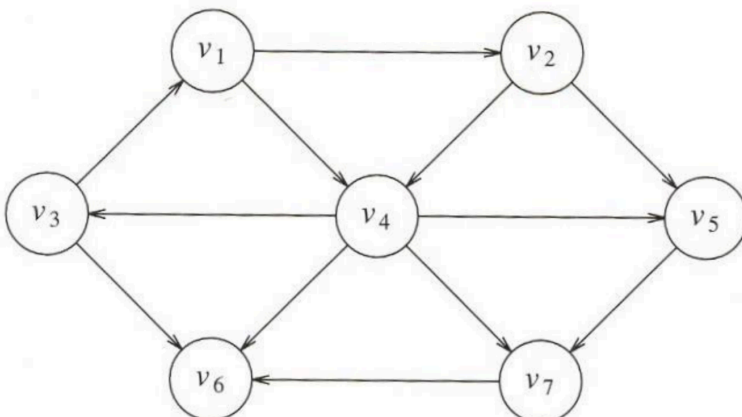
- **Dijkstra's Algorithm:**(อาจารย์สอน)
 - ใช้สำหรับกราฟที่ไม่มี weight ติดลบ
 - เลือกโหนดที่มีระยะทางสั้นที่สุดจากจุดเริ่มต้นแล้วอัปเดตระยะทางของโหนดข้างเคียง
- **Bellman-Ford Algorithm:**
 - รองรับกราฟที่มี weight ติดลบ
 - สามารถตรวจจับวงจรที่มีน้ำหนักรวมลบได้
- **Floyd-Warshall Algorithm:**
 - หาเส้นทางที่สั้นที่สุดระหว่างทุกคู่ของโหนด
 - ใช้ Dynamic Programming เพื่ออัปเดตระยะทาง

เนื้อหาในวิชา Shortest Path Algorithms:

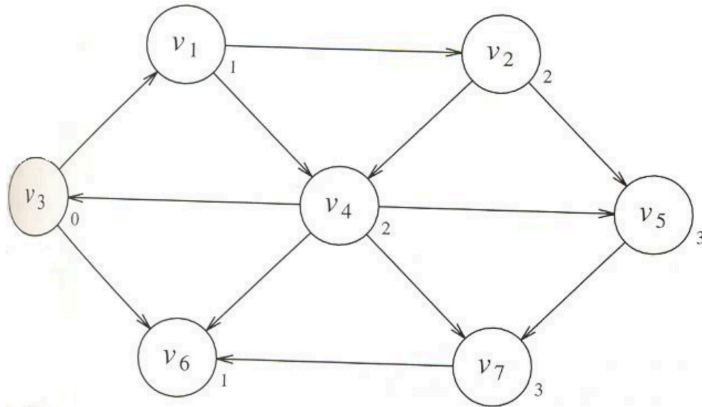
- Unweighted Shortest Paths
- Dijkstra's Algorithm

Unweighted Shortest Paths

เมื่ออาจารย์ให้กราฟมาแต่ไม่มี weight



การคำนวณ weight โดยกำหนด $s = v_3$



การหาเส้นทางที่สั้นที่สุด

D_v ย่อมาจาก คำว่า distance แปลว่า ระยะทาง 999

P_v ย่อมาจาก คำว่า path แปลว่า เส้นทาง 0

known หมายถึง node ที่เราเคยไปเยี่ยมชมหรือเราคำนวณแล้ว

เริ่มจากตั้งระยะทางเริ่มต้นเป็น 0 สำหรับ node ต้นทาง (1) และ ∞ (Infinity) สำหรับทุก node อื่น
ทำเครื่องหมาย node ทั้งหมดว่ายังไม่ได้เยี่ยมชม (known = F)

v	Initial State			v_3 Dequeued			v_1 Dequeued			v_6 Dequeued		
	known	d_v	p_v	known	d_v	p_v	known	d_v	p_v	known	d_v	p_v
v_1	F	∞	0	F	1	v_3	T	1	v_3	T	1	v_3
v_2	F	∞	0	F	∞	0	F	2	v_1	F	2	v_1
v_3	F	0	0	T	0	0	T	0	0	T	0	0
v_4	F	∞	0	F	∞	0	F	2	v_1	F	2	v_1
v_5	F	∞	0	F	∞	0	F	∞	0	F	∞	0
v_6	F	∞	0	F	1	v_3	F	1	v_3	T	1	v_3
v_7	F	∞	0	F	∞	0	F	∞	0	F	∞	0
Q :	v_3			v_1, v_6			v_6, v_2, v_4			v_2, v_4		

v	v_2 Dequeued			v_4 Dequeued			v_5 Dequeued			v_7 Dequeued		
	known	d_v	p_v	known	d_v	p_v	known	d_v	p_v	known	d_v	p_v
v_1	T	1	v_3	T	1	v_3	T	1	v_3	T	1	v_3
v_2	T	2	v_1	T	2	v_1	T	2	v_1	T	2	v_1
v_3	T	0	0	T	0	0	T	0	0	T	0	0
v_4	F	2	v_1	T	2	v_1	T	2	v_1	T	2	v_1
v_5	F	3	v_2	F	3	v_2	T	3	v_2	T	3	v_2
v_6	T	1	v_3	T	1	v_3	T	1	v_3	T	1	v_3
v_7	F	∞	0	F	3	v_4	F	3	v_4	T	3	v_4
Q :	v_4, v_5			v_5, v_7			v_7			empty		

แสดงวิธีทำแต่ละ step และทำให้ known เป็น T ให้หมด

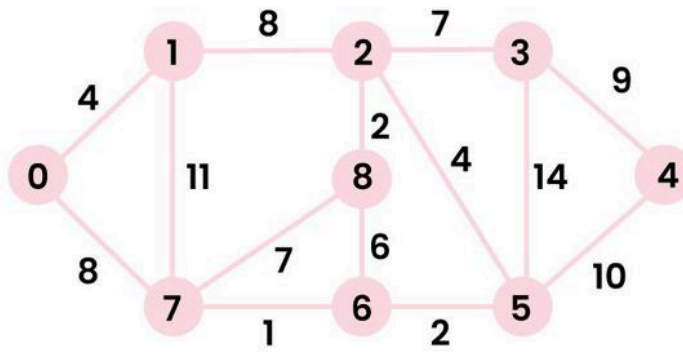
ถาม $v_3 \rightarrow v_7$ ใช้เส้นทางอะไร

ตอบ: $v_7 \rightarrow v_4 \rightarrow v_1 \rightarrow v_3 = v_3 \rightarrow v_1 \rightarrow v_4 \rightarrow v_7$ ระยะทาง = 3

Dijkstra's Algorithm

ทำเหมือน unweighted แต่ d แต่ละ node = weight ที่วิ่งมา

ตัวอย่าง



Working of Dijkstra's Algorithm



ในการทำ Dijkstra's Algorithm เพื่อนำทางจาก node 1 ไป node 5 จะต้องคำนวณระยะทางที่สั้นที่สุดจาก node เริ่มต้นไปยังทุก node อื่น ๆ อย่างเป็นขั้นตอน ในแต่ละขั้นตอน เราจะอัปเดตระยะทางที่น้อยที่สุดจนกว่าจะถึงปลายทาง

ตารางเริ่มต้น:

Node	ระยะทางจาก Node 1	Node ก่อนหน้า	สถานะ
0	∞	-	F
1	0	-	T
2	∞	-	F
3	∞	-	F
4	∞	-	F
5	∞	-	F
6	∞	-	F
7	∞	-	F
8	∞	-	F

1. เริ่มต้นที่ Node 1:

- Node 1 เชื่อมกับ Node 2 (ระยะทาง 8) และ Node 7 (ระยะทาง 11)
- อัปเดตระยะทางสำหรับ Node 2 และ 7

Node	ระยะทางจาก Node 1	Node ก่อนหน้า	สถานะ
0	∞	-	F
1	0	-	T
2	8	1	F
3	∞	-	F
4	∞	-	F
5	∞	-	F
6	∞	-	F
7	11	1	F
8	∞	-	F

2. เชื่อมชม Node 2 (ระยะทาง 8):

- Node 2 เชื่อมกับ Node 3 (ระยะทาง 7) และ Node 8 (ระยะทาง 2)
- อัปเดตระยะทางสำหรับ Node 3 และ Node 8

Node	ระยะทางจาก Node 1	Node ก่อนหน้า	สถานะ
0	∞	-	F
1	0	-	T
2	8	1	T
3	15	2	F
4	∞	-	F
5	∞	-	F
6	∞	-	F
7	11	1	F
8	10	2	F

3. เชื่อมชม Node 7 (ระยะทาง 11):

- Node 7 เชื่อมกับ Node 6 (ระยะทาง 1) และ Node 8 (ระยะทาง 7)
- อัปเดตระยะทางสำหรับ Node 6

Node	ระยะทางจาก Node 1	Node ก่อนหน้า	สถานะ
0	∞	-	F
1	0	-	T
2	8	1	T
3	15	2	F
4	∞	-	F
5	∞	-	F
6	12	7	F
7	11	1	T
8	10	2	F

4. เชื่อมชม Node 8 (ระยะทาง 10):

- Node 8 เชื่อมกับ Node 6 (ระยะทาง 6) และ Node 5 (ระยะทาง 4)
- อัปเดตระยะทางสำหรับ Node 5

Node	ระยะทางจาก Node 1	Node ก่อนหน้า	สถานะ
0	∞	-	F
1	0	-	T
2	8	1	T
3	15	2	F

Node	ระยะทางจาก Node 1	Node ก่อนหน้า	สถานะ
4	∞	-	F
5	14	8	F
6	12	7	F
7	11	1	T
8	10	2	T

5. เยี่ยมชม Node 6 (ระยะทาง 12):

- Node 6 เชื่อมกับ Node 5 (ระยะทาง 2)
- อัปเดตระยะทางสำหรับ Node 5 เป็น 14

Node	ระยะทางจาก Node 1	Node ก่อนหน้า	สถานะ
0	∞	-	F
1	0	-	T
2	8	1	T
3	15	2	F
4	∞	-	F
5	14	6	F
6	12	7	T
7	11	1	T
8	10	2	T

- ระยะทางรวม: 14
- เส้นทาง: $1 \rightarrow 2 \rightarrow 8 \rightarrow 6 \rightarrow 5$

Graph

วิธีทำ:

- การแทนกราฟ:
 - **Adjacency List:** ใช้ลิสต์เก็บ neighbor ของแต่ละโหนด (ประหยัดพื้นที่)
 - **Adjacency Matrix:** ใช้เมทริกซ์สองมิติเก็บการเชื่อมต่อระหว่างโหนด (ง่ายต่อการตรวจสอบการเชื่อมต่อ)

- การ Traversal:

- **DFS (Depth-First Search):** ใช้สแต็กหรือการเรียกแบบ recursive เพื่อสำรวจโหนด
- **BFS (Breadth-First Search):** ใช้คิวในการสำรวจโหนดในระดับที่กว้างขึ้น

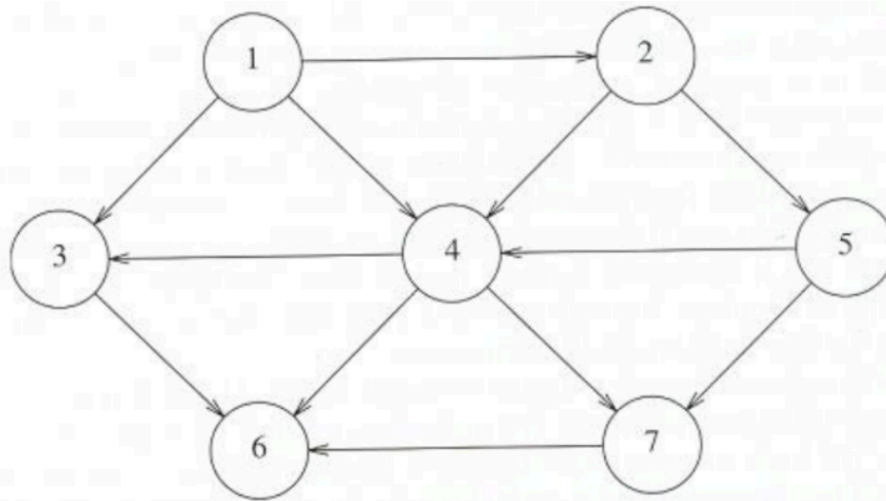
การใช้งาน:

- การวิเคราะห์เครือข่ายสังคม, การค้นหาเส้นทางในแผนที่, การจัดการโครงสร้างข้อมูลอื่น ๆ

กราฟ (Graph) คือ ฎุเพิล $G = (V, E)$:

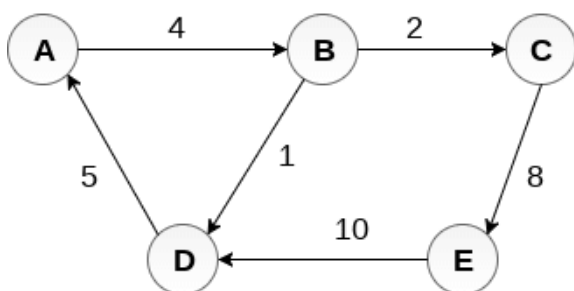
- V : เซตของ **จุดยอด** (vertices) หรือ **โหนด** (nodes)
- E : เซตของ **เส้นเชื่อม** (edges)
 - เส้นเชื่อมแต่ละเส้นคือคู่อันดับ (v, w) โดยที่ $v, w \in V$
- หากคู่อันดับจุดยอดเป็นลำดับ (ordered) กราฟนั้นเรียกว่า **กราฟมีทิศทาง** (directed)

- กราฟมีทิศทางบางครั้งเรียกว่า **กราฟทิศทาง** (digraphs)
- จุดยอด w **ติดกัน** (adjacent) กับ v ก็ต่อเมื่อ $(v, w) \in E$
หมายเหตุเพิ่มเติม:
- เราจะพิจารณาเฉพาะ **กราฟมีทิศทาง**
- จุดยอดจะถูกกำหนดหมายเลขโดยเริ่มที่ 1
- กราฟด้านล่างมี 7 จุดยอด และ 12 เส้นเชื่อม



การแทนด้วยเมทริกซ์การเชื่อมต่อ (Adjacency Matrix Representation)

- ใช้อาร์เรย์สองมิติ (two-dimensional array) ในการแทนกราฟ
 - สำหรับแต่ละเส้นเชื่อม $(u, v) \in E$ เรากำหนด $A[u][v] = 1$
 - $A[u][v] = 0$ ถ้าไม่มีเส้นเชื่อม
- ถ้าเส้นเชื่อมมีค่าน้ำหนัก (weight) ให้กำหนด $A[u][v] = weight$
 - ใช้ $-\infty$ หรือ ∞ เพื่อแสดงว่าไม่มีเส้นเชื่อม
- **การใช้หน่วยความจำ (Space Requirement):** $\theta(|V|^2)$
- เมทริกซ์ติดกันเหมาะสมที่จะใช้ในกรณีที่กราฟมีความหนาแน่น (dense)
 - เมื่อจำนวนเส้นเชื่อม $|E| = \theta(|V|^2)$



Weighted Directed Graph

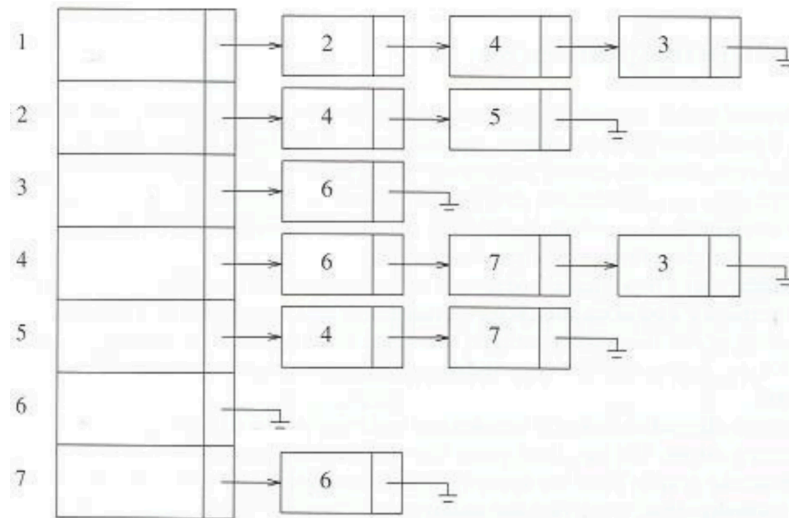
	A	B	C	D	E
A	0	4	0	0	0
B	0	0	2	1	0
C	0	0	0	0	8
D	5	0	0	0	0
E	0	0	0	10	0

Adjacency Matrix

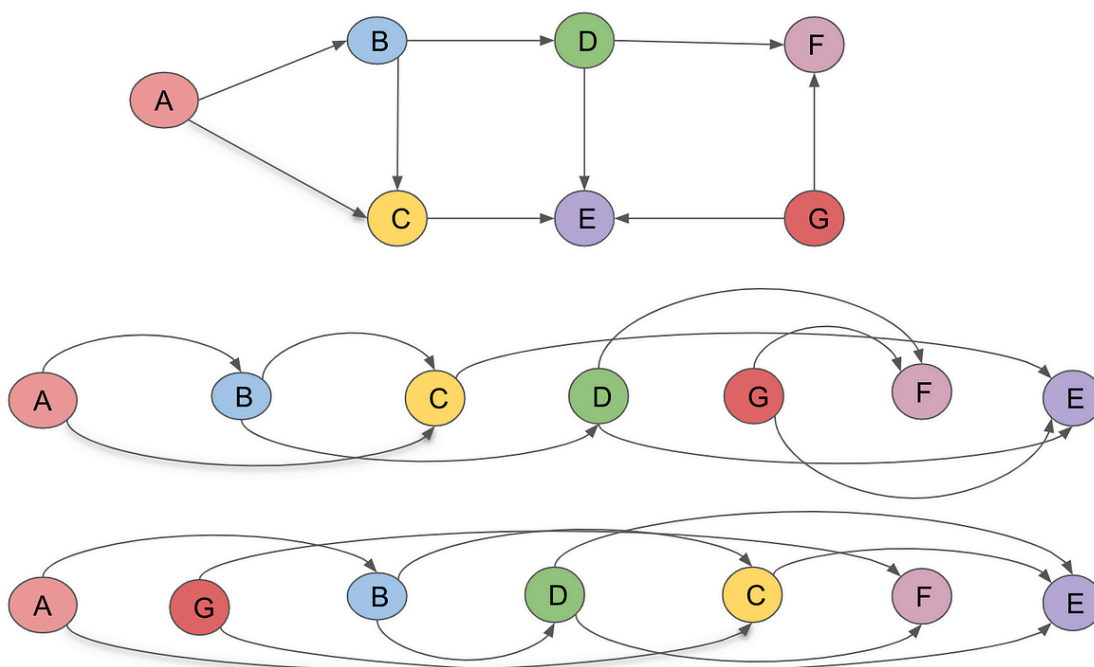
การแทนด้วยรายการการเชื่อมต่อ (Adjacency List Representation)

- หากกราฟ **ไม่หนาแน่น** (เป็น **กราฟบาง**) วิธีที่ดีกว่าคือการแทนด้วย **รายการการเชื่อมต่อ**
- สำหรับแต่ละจุดยอด เราจะเก็บรายการของจุดยอดที่อยู่ติดกัน
- ความต้องการพื้นที่คือ $O(|E| + |V|)$
 - ตัวอย่าง: $12 + 7 = 19$

การแทนกราฟ (Representation of Graphs)



Topological Sort



Topological Sort

แนวคิดหลัก:

- การจัดลำดับโหนดในกราฟที่เป็นกราฟทิศทางและไม่มียังงจร (DAG) โดยให้ถ้าโหนด A ชี้ไปยังโหนด B, A จะมาอยู่ก่อน B ในลำดับ

วิธีทำ:

- Kahn's Algorithm:**
 - หาโหนดที่ไม่มีขาเข้า (in-degree = 0)
 - นำโหนดเหล่านั้นมาใส่ในลำดับ
 - ลบโหนดเหล่านั้นและขาออกจากกราฟ
 - ทำซ้ำจนกว่าจะไม่มีโหนดเหลือ
- DFS-based Algorithm:**
 - ทำ DFS บนกราฟ

2. เมื่อจบการสำรวจโหนด ให้ใส่โหนดนั้นเข้าไปในสแต็ก
3. นำโหนดจากสแต็กออกมาเป็นลำดับที่ออกไปล่อจิก

การใช้งาน:

- การจัดลำดับงานที่ต้องทำตามลำดับ, การคอมไพล์โปรแกรมที่มี dependencies

โค้ด

```
#include <iostream>
#include <vector>
#include <queue>
#include <stdexcept>

class Graph {
public:
    Graph(int numVertices);
    void addEdge(int from, int to);
    void topsort();

private:
    int NUM_VERTICES;
    std::vector<std::vector<int>> adjList; // Adjacency list
    std::vector<int> indegree;           // In-degree of each vertex
};

Graph::Graph(int numVertices)
    : NUM_VERTICES(numVertices), adjList(numVertices, std::vector<int>()),
    indegree(numVertices, 0) {}

void Graph::addEdge(int from, int to) {
    adjList[from].push_back(to);
    indegree[to]++;
}

void Graph::topsort() {
    std::queue<int> q;
    int counter = 0;
    std::vector<int> topNum(NUM_VERTICES, 0);

    // Enqueue vertices with in-degree 0
    for (int v = 0; v < NUM_VERTICES; ++v) {
        if (indegree[v] == 0) {
            q.push(v);
        }
    }

    while (!q.empty()) {
        int v = q.front();
        q.pop();
        topNum[v] = ++counter; // Assign next number

        // Iterate through all adjacent vertices
        for (int w : adjList[v]) {
            indegree[w]--;
            if (indegree[w] == 0) {
                q.push(w);
            }
        }
    }
}
```

```

        if (--indegree[w] == 0) {
            q.push(w);
        }
    }
}

// Check if there was a cycle
if (counter != NUM_VERTICES) {
    throw std::runtime_error("Cycle detected in the graph");
}

// Optional: Print topological order
std::cout << "Topological Order: ";
for (int v = 0; v < NUM_VERTICES; ++v) {
    std::cout << v << " ";
}
std::cout << std::endl;
}

// Example usage
int main() {
    int numVertices = 6;
    Graph g(numVertices);

    // Adding edges
    g.addEdge(5, 2);
    g.addEdge(5, 0);
    g.addEdge(4, 0);
    g.addEdge(4, 1);
    g.addEdge(2, 3);
    g.addEdge(3, 1);

    try {
        g.topsort();
    } catch (const std::runtime_error& e) {
        std::cerr << e.what() << std::endl;
    }

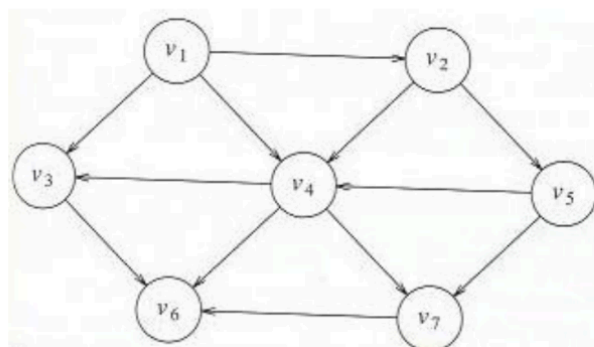
    return 0;
}

```

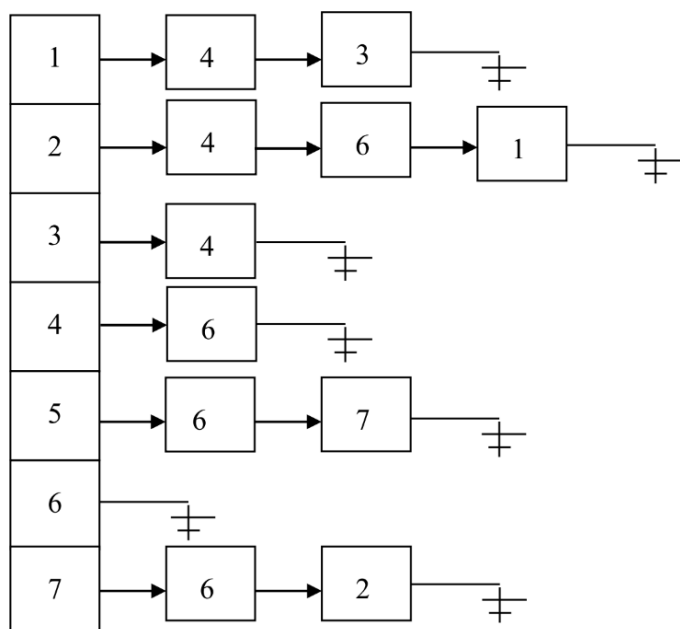
ตัวอย่าง 1:

Result of Applying topological sort to the graph below

Vertex	Indegree Before Dequeue #						
	1	2	3	4	5	6	7
v_1	0	0	0	0	0	0	0
v_2	1	0	0	0	0	0	0
v_3	2	1	1	1	0	0	0
v_4	3	2	1	0	0	0	0
v_5	1	1	0	0	0	0	0
v_6	3	3	3	3	2	1	0
v_7	2	2	2	1	0	0	0
Enqueue	v_1	v_2	v_5	v_4	v_3, v_7		v_6
Dequeue	v_1	v_2	v_5	v_4	v_3	v_7	v_6



ตัวอย่างปilhaแล้ว



เขียน Indegree Array สำหรับคำนวณ Topology Sort

v	1	2	3	4	5	6	7
1	1	1	1	0			
2	1	1	0				
3	1	1	1	1	0		
4	3	3	3	2	1	0	

v	1	2	3	4	5	6	7
5	0						
6	4	3	2	1	1	1	0
7	1	0					
Enqueue	5	7	2	1	3	4	6
Dequeue	5	7	2	1	3	4	6

จะได้ผลลัพธ์ Topologogy Sort จากการ Dequeue แต่ละตัวเป็น

```
5, 7, 2, 1, 3, 4, 6
```

Sorting

การเรียงลำดับเลขใน array

3 กรณีที่สามารถเกิดขึ้นในทุก algorithms

- Best Case(ดีที่สุด)
- Worst Case(แย่ที่สุด)
- Average Case(โดยปกติ)

algorithms

- 1. Insertion Sort
- 2. Shellsort

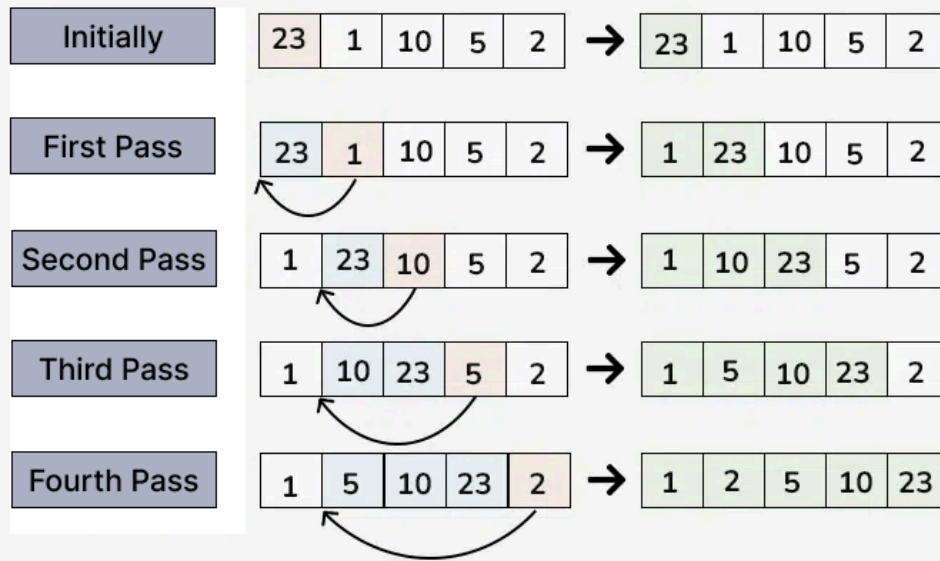
1. InsertSort

ตัวอย่าง 1

Original	34	8	64	51	32	21	P. Moved

After $p = 1$	8	34	64	51	32	21	1
After $p = 2$	8	34	64	51	32	21	0
After $p = 3$	8	34	51	64	32	21	1
After $p = 4$	8	32	34	51	64	21	3
After $p = 5$	8	21	32	34	51	64	4

ตัวอย่าง 2



Insertion Sort



โค้ดการ Insertion Sort ใน c++

```
// Insertion sort routine
template <class Comparable>
void insertionSort(vector<Comparable> &a) {
    for (int p = 1; p < a.size(); p++) {
        Comparable tmp = a[p];
        int j;

        for (j = p; j > 0 && tmp < a[j - 1]; j--)
            a[j] = a[j - 1];

        a[j] = tmp;
    }
}
```

2. Shellsort

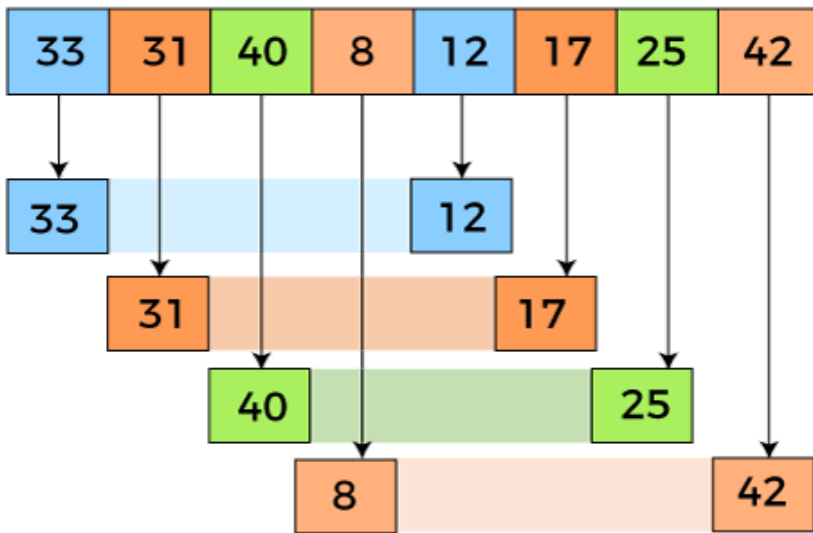
shellsort เป็นการเรียงข้อมูลโดยเอา $gap = n/2$ และทำการเรียงและวนไปเรื่อยๆจน $gap = 1$ แล้วจะไปใช้ insert sort ตามปกติ

ตัวอย่าง 1

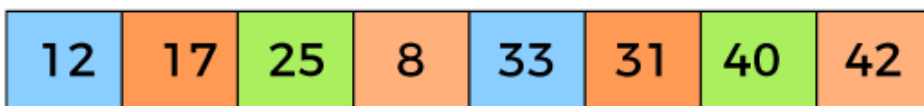
33, 31, 40, 8, 12, 17, 25, 42

โดยเราจะเริ่มจากการคำนวณโดยในที่นี้ $n=8$ ก็จะได้เป็น $gap = 8/2 = 4$

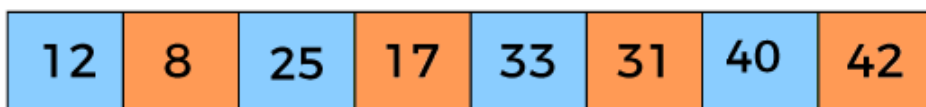
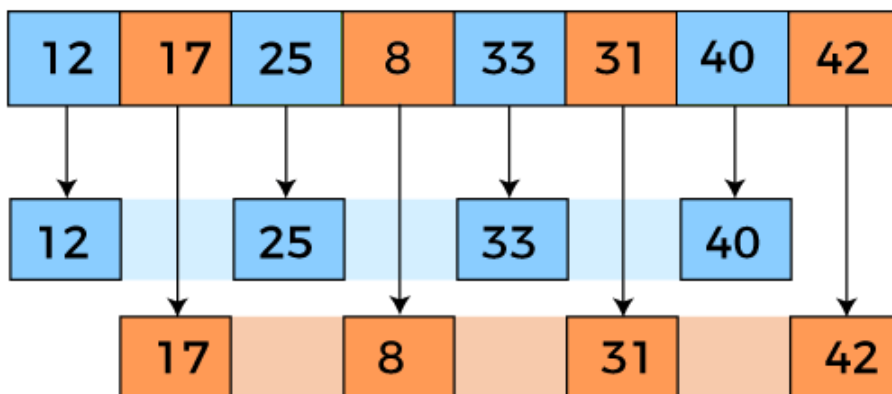
ต่อไปมาจับทำ shellsort ต่อตามรูป



จะสังเกตว่าเราแบ่งข้อมูลได้ 2 ชุดชุดละ 4 ค่าซึ่งได้ตาม gap ที่เรากำหนดเลย



นำ gap ไปปรับสมการต่อก็จะได้ $gap = 4/2 = 2$ ก็จะแบบข้อมูลได้ชุดละ 2 ค่า



ต่อไปนำ gap ไปหารต่อ $gap = 2/2 = 1$ เมื่อเป็น 1 ให้เราไปทำ insertion sort และได้ผลลัพธ์

12	8	25	17	33	31	40	42
12	8	25	17	33	31	40	42
8	12	25	17	33	31	40	42
8	12	25	17	33	31	40	42
8	12	17	25	33	31	40	42
8	12	17	25	33	31	40	42
8	12	17	25	33	31	40	42
8	12	17	25	31	33	40	42
8	12	17	25	31	33	40	42
8	12	17	25	31	33	40	42

ตอบ: 8, 12, 17, 25, 31, 33, 40, 42

โค้ดการ shellsort โดยอาจารย์สุดตั้งแต่ทำโมแกเอา $n/2.6$ ว่า 555

```
#include <vector>

template <class Comparable>
void shellsort(std::vector<Comparable> &a) {
    int j;
    for (int gap = a.size() / 2.6; gap > 0; gap = (gap == 2) ? 1 : (gap / 2)) {
        for (int i = gap; i < a.size(); i++) {
            Comparable tmp = a[i];
            for (j = i; j >= gap && tmp < a[j - gap]; j -= gap) {
                a[j] = a[j - gap];
            }
            a[j] = tmp;
        }
    }
}
```

3. Quick Sort

```
int partition(vector<int>& arr, int low, int high) {

    // Choose the pivot
    int pivot = arr[high];

    // Index of smaller element and indicates
```

```

// the right position of pivot found so far
int i = low - 1;

// Traverse arr[low..high] and move all smaller
// elements on left side. Elements from low to
// i are smaller after every iteration
for (int j = low; j <= high - 1; j++) {
    if (arr[j] < pivot) {
        i++;
        swap(arr[i], arr[j]);
    }
}

// Move pivot after smaller elements and
// return its position
swap(arr[i + 1], arr[high]);
return i + 1;
}

// The QuickSort function implementation
void quickSort(vector<int>& arr, int low, int high) {

    if (low < high) {

        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);

        // Recursion calls for smaller elements
        // and greater or equals elements
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

```

Step 1
Determine pivot

4	2	6	5	3	9
---	---	---	---	---	---

Step 2
Start pointers at left and right

4	2	6	5	3	9
↑ L				↑ R	

Step 3
Since $4 < 5$, shift left pointer

4	2	6	5	3	9
↑ L				↑ R	

Step 4
Since $2 < 5$, shift left pointer
Since $6 > 5$, stop

4	2	6	5	3	9
↑ L				↑ R	

Step 5
Since $9 > 5$, shift right pointer
Since $3 < 5$, stop

4	2	6	5	3	9
↑ L				↑ R	

Step 6
Swap values at pointers

4	2	3	5	6	9
↑ L				↑ R	

Step 7
Move pointers one more step

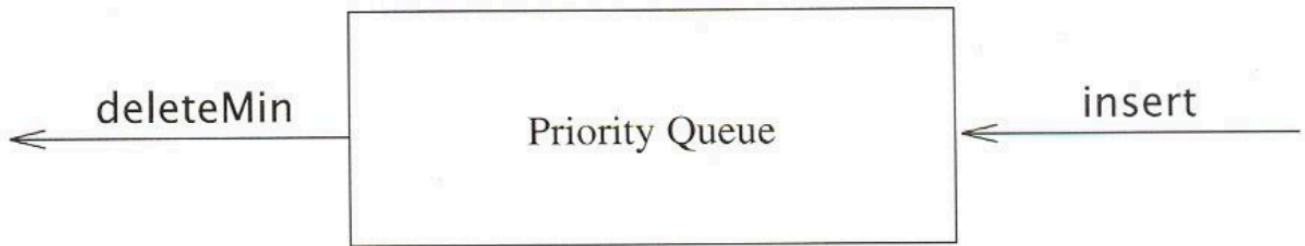
4	2	3	5	6	9
			↑ L	↑ R	

Step 8
Since $5 == 5$,
move pointers one more step
Stop

4	2	3	5	6	9
			↑ R	↑ L	

Heap

Heap คือตัวที่จะเอาไปสารต่อทำ priority queue



แนวคิดหลัก:

- โครงสร้างข้อมูลแบบ Heap เป็นต้นไม้แบบเต็ม (complete binary tree) ที่แต่ละโหนดมีค่ามากกว่า (max-heap) หรือ น้อยกว่า (min-heap) โหนดลูกทุกโหนด
- Priority Queue ใช้ Heap ในการจัดลำดับความสำคัญของข้อมูล

วิธีทำ:

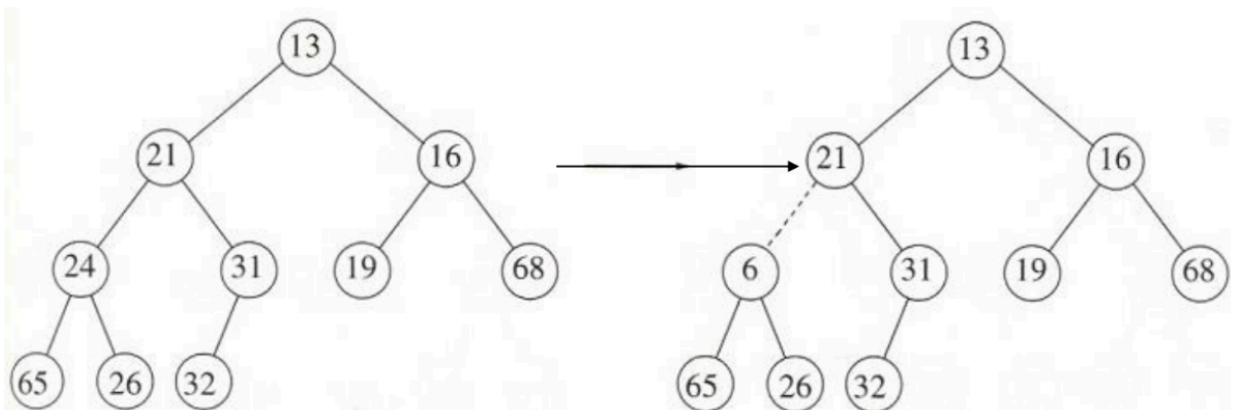
- Insertion:** แทรกข้อมูลที่ด้านท้ายของ Heap แล้วปรับตำแหน่งขึ้น (Heapify Up)
- Extract-Max/Min:** นำค่าที่สูงสุดหรือต่ำสุดออก แล้วย้ายค่าจากท้ายขึ้นมาใหม่ แล้วปรับตำแหน่งลง (Heapify Down)

การใช้งาน:

- การจัดการคิวที่ต้องการลำดับความสำคัญ เช่น การจัดการงานในระบบปฏิบัติการ, อัลกอริทึมการค้นหาเส้นทาง

คุณสมบัติของ Heap

- สำหรับทุกโหนด X คีย์ในโหนดพารেন্ট parent จะมีค่าน้อยกว่าคีย์ใน X (ยกเว้น root)
- องค์ประกอบที่มีค่าน้อยที่สุดจะสามารถพบได้ที่ root เสมอ



Only left tree is a heap !

๕

โค้ดของ Heap

```
// Class interface for priority queues
template <class Comparable>
class BinaryHeap {
public:
    explicit BinaryHeap(int capacity = 100);

    bool isEmpty() const;
    bool isFull() const;
```

```

    const Comparable& findMin() const;

    void insert(const Comparable& x);
    void deleteMin();
    void deleteMin(Comparable& minItem);
    void makeEmpty();

private:
    int currentSize;           // Number of elements in heap
    std::vector<Comparable> array; // The heap array

    void buildHeap();
    void percolateDown(int hole);
};

/* Construct the binary heap. */
template <class Comparable>
BinaryHeap<Comparable>::BinaryHeap(int capacity)
    : array(capacity + 1), currentSize(0) { }

/* Test if the priority queue is logically empty. */
template <class Comparable>
bool BinaryHeap<Comparable>::isEmpty() const {
    return currentSize == 0;
}

/* Test if the priority queue is logically full. */
template <class Comparable>
bool BinaryHeap<Comparable>::isFull() const {
    return currentSize == array.size() - 1;
}

/* Make the priority queue logically empty. */
template <class Comparable>
void BinaryHeap<Comparable>::makeEmpty() {
    currentSize = 0;
}

/* Insert item x into the priority queue. */
template <class Comparable>
void BinaryHeap<Comparable>::insert(const Comparable& x) {
    if (isFull())
        throw std::overflow_error("Overflow: Priority queue is full.");

    // Percolate up
    int hole = ++currentSize;
    for (; hole > 1 && x < array[hole / 2]; hole /= 2) {
        array[hole] = array[hole / 2];
    }
    array[hole] = x;
}

/* Remove the smallest item from the priority queue. */
template <class Comparable>
void BinaryHeap<Comparable>::deleteMin(Comparable& minItem) {

```

```

    if (isEmpty())
        throw std::underflow_error("Underflow: Priority queue is empty.");

    minItem = array[1];
    array[1] = array[currentSize--];
    percolateDown(1);
}

/* Internal method to percolate down in the heap. */
template <class Comparable>
void BinaryHeap<Comparable>::percolateDown(int hole) {
    int child;
    Comparable tmp = array[hole];

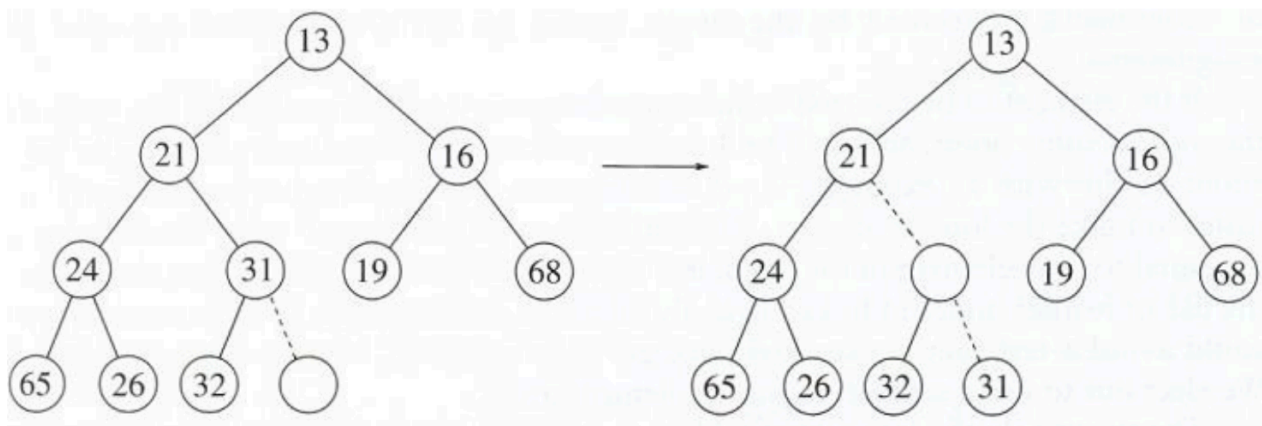
    for (; hole * 2 <= currentSize; hole = child) {
        child = hole * 2;
        if (child != currentSize && array[child + 1] < array[child]) {
            child++;
        }
        if (array[child] < tmp) {
            array[hole] = array[child];
        } else {
            break;
        }
    }
    array[hole] = tmp;
}

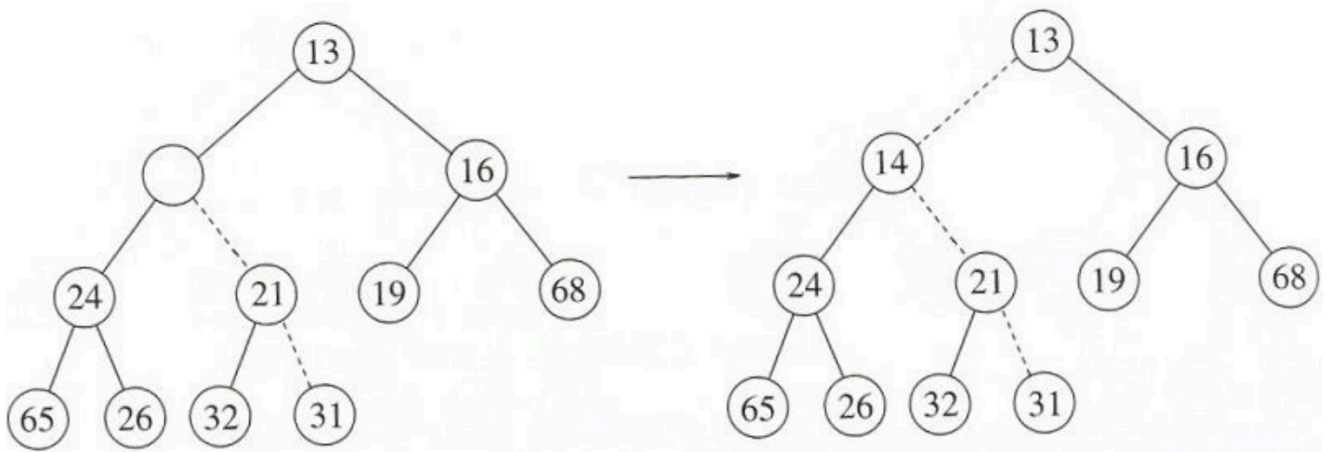
/* Establish heap order property. */
template <class Comparable>
void BinaryHeap<Comparable>::buildHeap() {
    for (int i = currentSize / 2; i > 0; i--) {
        percolateDown(i);
    }
}

```

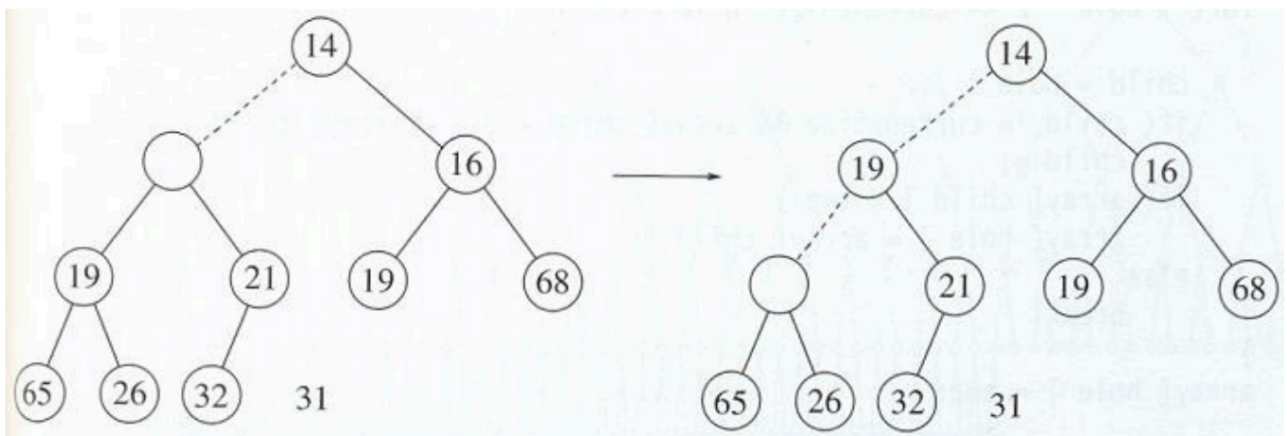
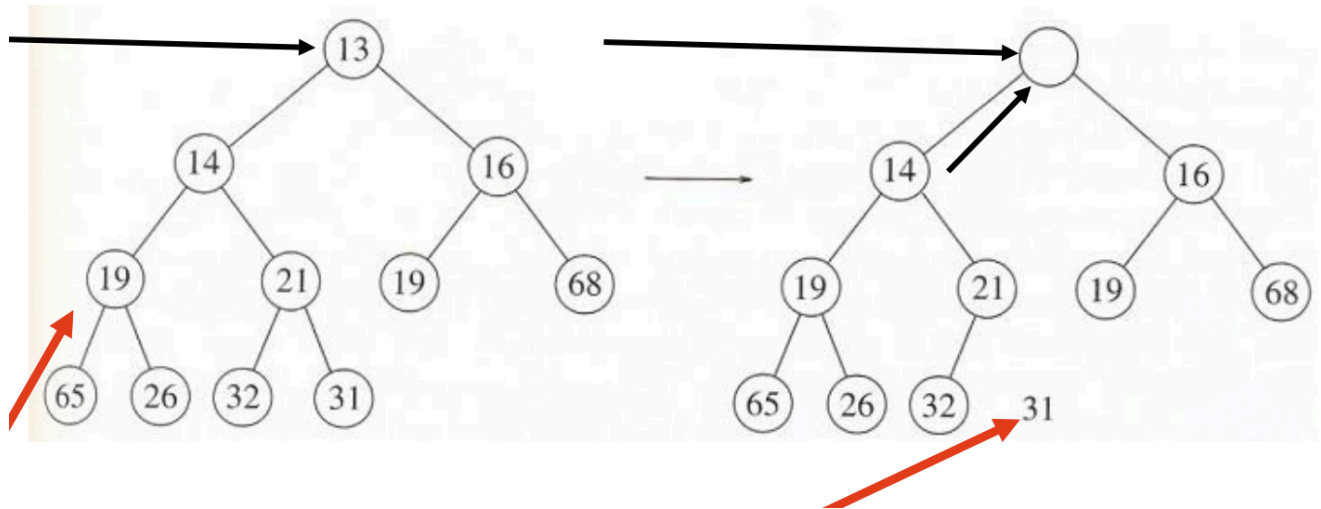
ตัวอย่าง Heap

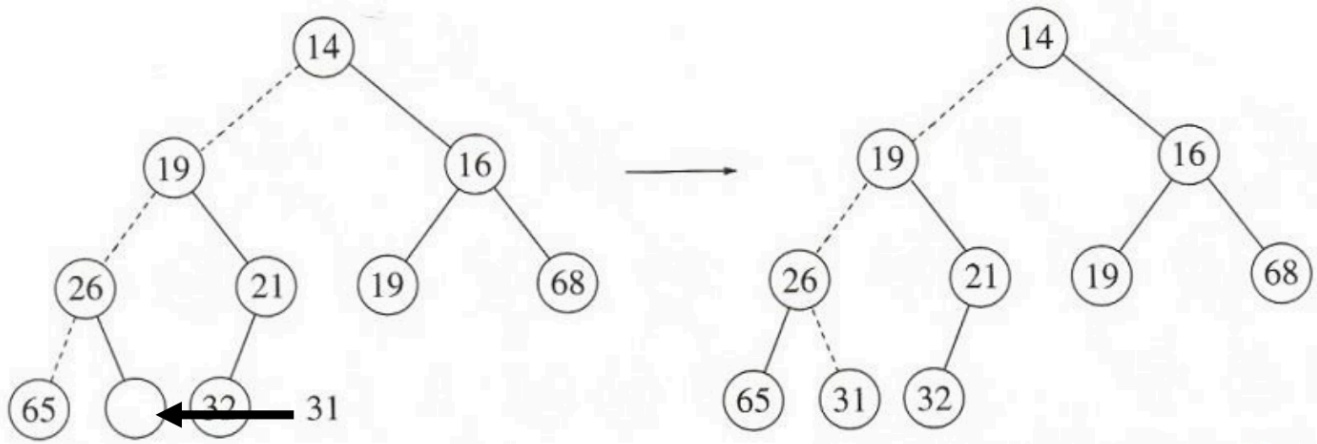
insert(14)





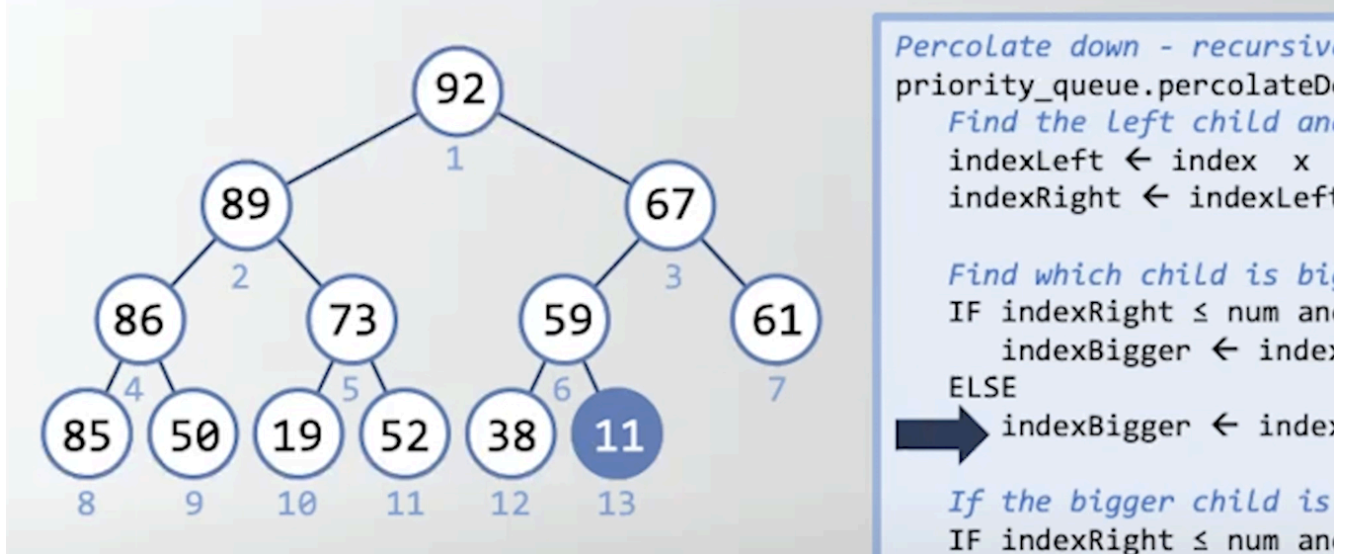
DeleteMin()



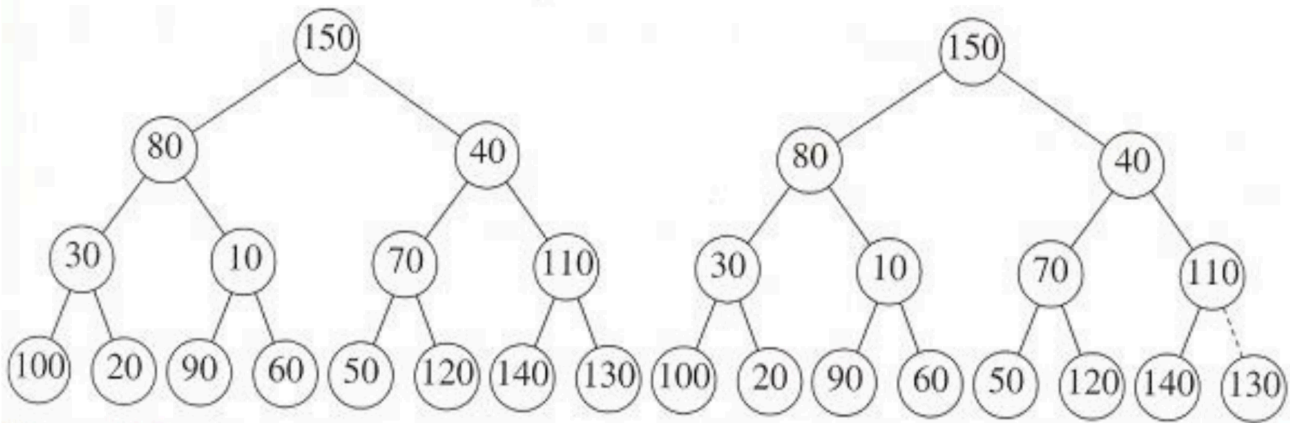


percolateDown

Call percolateDown(13) corresponding to the value 11.



Left : initial heap; right: after percolateDown(7)



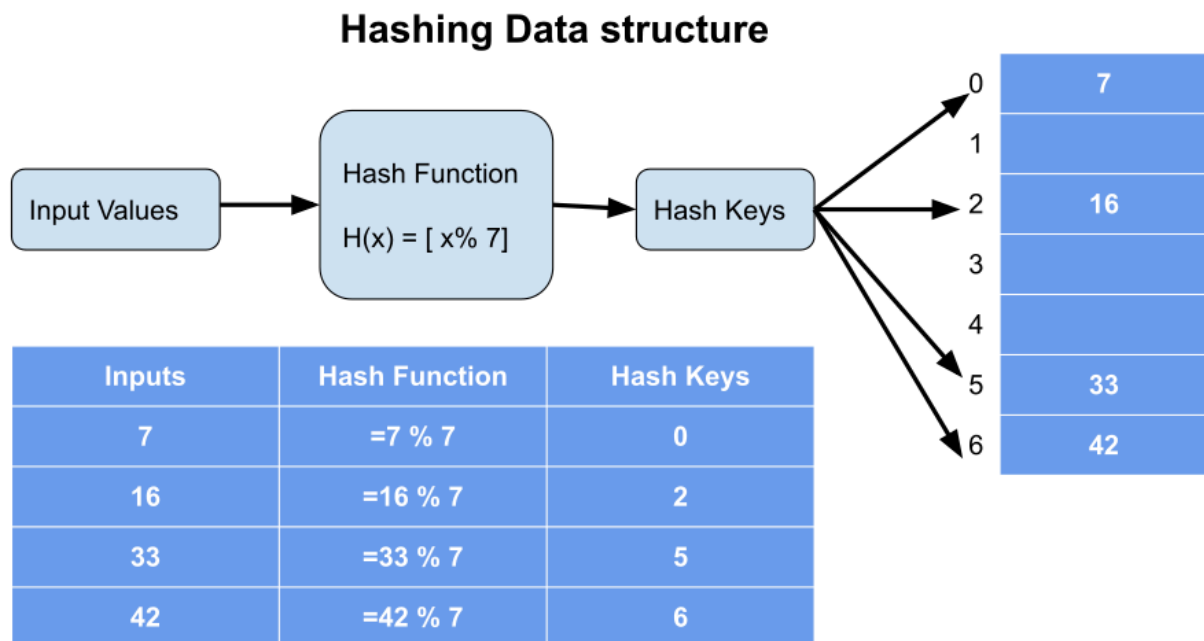
Hashing

ในบทนี้มี 2 algorithms

1. ที่อาจารย์สอนแล้วน่าจะออก(Double Hashing)
2. ที่อาจารย์สอนแต่ไม่น่าจะออก

เมื่ออาจารย์ให้เขียนตารางเขียนช่องที่มีคำว่า active ถ้าไม่มี เขียน empty

1. ที่อาจารย์จะออก(Double Hashing)



สมการที่จำเป็น

คำนวณแฮช

```
h0 = key % table_size
Pseudorandomness```
คำนวณหาขนาด

```js
h0 = key % table_size

// prime จำนวนเฉพาะที่ค่าใกล้ table_size ที่สุด
// n คือจำนวนครั้งที่ชนเริ่มด้วย 1
f(n) = n * (prime - (h0 % prime))
```

ตัวอย่าง

```
h0 = key % table_size
shell sort โดยอาจารย์สุดตั้งแต่ทำโมเคเอา n/2.6 วั

// ตัวอย่างชนรอบที่ 1
f1 = 1 * (prime - (h0 % prime))
h1 = (f1 + h0) % table_size

// ถ้าวชนอีก
f2 = 2 * (prime - (h0 % prime))
h2 = (f2 + h0) % table_size
```

# ตัวอย่าง 1

Input 1 array ตัวเลข

```
[10, 20, 31, 4, 15, 28, 17, 88, 59]
```

Input 2 ขนาดตาราง(table size)

```
10
```

ทำ array เก็บ table hash ที่คำนวณแล้วไว้ว่า

```
tables = []
```

ขั้นแรกให้นำ  $\text{key} \bmod \text{table\_size}$  จะได้  $h_1$

```
h0 = key % table_size // 10 % 10 = 0
```

แล้วนำผลลัพธ์ไปใส่ใน table

```
tables[h0] = key
```

[10, -, -, -, -, -, -, -]

ถ้าเช่นเช่น  $\text{key} = 20$

```
h0 = 20 % 10 // = 0

// แต่มี 10 อยู่ใน index ที่ 0 อยู่แล้วให้เราคิดการชน
// การชนครั้งแรก
prime = 7 // เพราะจำนวนเฉพาะ 7 ใกล้เคียง 10 ที่สุด

f1 = 1 * (prime - (h0 % prime)) // 1 * (7 - (0 % 7)) = 7
h1 = (f1 + h0) % table_size // (7 + 0) % 10 = 7

// จะได้ 7 ดูว่า index ที่ 7 มีค่ายังถ้าไม่มีใส่ได้เลย
tables[h1] = key
```

[10, 20, -, -, -, -, -, -]

ทำต่อเองนะไอน้อง

## ตารางจำนวนเฉพาะ

	2	3	5	7	11	13	17	19	23
29	31	37	41	43	47	53	59	61	67
71	73	79	83	89	97	101	103	107	109
113	127	131	137	139	149	151	157	163	167
173	179	181	191	193	197	199	211	223	227
229	233	239	241	251	257	263	269	271	277
281	283	293	307	311	313	317	331	337	347
349	353	359	367	373	379	383	389	397	401
409	419	421	431	433	439	443	449	457	461
463	467	479	487	491	499	503	509	521	523
541	547	557	563	569	571	577	587	593	599
601	607	613	617	619	631	641	643	647	653
659	661	673	677	683	691	701	709	719	727
733	739	743	751	757	761	769	773	787	797
809	811	821	823	827	829	839	853	857	859
863	877	881	883	887	907	911	919	929	937
941	947	953	967	971	977	983	991	997	

## 2. ที่อาจารย์ไม่น่าจะออก

**Linear Probing** is used to resolve collision

Problem: It is not easy to delete an element

- May have caused a collision before
- Mark the element deleted

Problem: Primary Clustering การเกาะกลุ่ม

$h_0 = \text{key} \% \text{table\_size}$

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1					58	58
2						69
3						
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

### Insert keys

**{89, 18, 49, 58, 69}**

- When 49 is inserted collision occurs
  - Put into the next available spot 0
- 58 collides with 18, 89, 49

11

### Quadratic Probing

$F(i)$  is a quadratic function probing is used to resolve collision

Ex :  $F(i) = i^2$

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1						
2					58	58
3						69
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

■ When 49 collides with 89, next position attempted is one cell away

■ 58 collides at position 8. The cell one away is tried, another collision occurs. It is inserted into the cell  $2^2=4$  away

### Rehashing

สมการที่จำเป็น

limit = percentage \* table\_size / 100

- If 23 is inserted, the table is over **70** percent full.

0	6
1	15
2	23
3	24
4	
5	
6	13



A new table is created

17 is the first prime twice as large as the old one; so

$$H_{\text{new}}(X) = X \bmod 17$$

0	
1	
2	
3	
4	
5	
6	6
7	23
8	24
9	
10	
11	
12	
13	13
14	
15	15
16	

## คำถามอื่นที่แล้ว

1. คุณสมบัติของ hash function ตามทฤษฎีในทางอุดมคติ มี 3 ข้อ คืออะไร

ตอบ: ทฤษฎีในอุดมคติ

ในการออกแบบและพัฒนาฟังก์ชันแฮช (Hash Function) ที่มีประสิทธิภาพสูงตามทฤษฎีในทางอุดมคติ ฟังก์ชันแฮชควรมีคุณสมบัติหลักๆ ดังต่อไปนี้:

1. **ความแน่นอน (Deterministic)**

- ฟังก์ชันแฮชควรให้ผลลัพธ์เดียวกันสำหรับอินพุตที่เหมือนกันทุกครั้ง กล่าวคือ หากมีการแฮชข้อมูลเดียวกันหลายครั้ง ผลลัพธ์ที่ได้จะต้องเหมือนกันเสมอ

2. **ประสิทธิภาพ (Efficiency)**

- ฟังก์ชันแฮชควรสามารถคำนวณค่าแฮชได้อย่างรวดเร็วและมีประสิทธิภาพ ไม่ว่าจะเป็นการประมวลผลข้อมูลขนาดเล็กหรือขนาดใหญ่

3. **ความต้านทานการย้อนกลับ (Pre-image Resistance)**

- ไม่ควรสามารถหาค่าอินพุตที่ให้ผลลัพธ์แฮชเฉพาะเจาะจงได้ในเวลาเชิงสมเหตุสมผล กล่าวคือ หากทราบค่าแฮชแล้ว ไม่สามารถหาข้อมูลต้นฉบับที่ให้ค่าแฮชนั้นได้

4. **ความต้านทานการหาค่าอินพุตอื่น (Second Pre-image Resistance)**

- หากมีอินพุตหนึ่งแล้ว ไม่ควรสามารถหาข้อมูลอื่นที่มีค่าแฮชเหมือนกันได้ในเวลาเชิงสมเหตุสมผล ซึ่งหมายความว่าไม่สามารถหาค่าอินพุตอื่นที่แตกต่างแต่ให้ค่าแฮชเดียวกันได้ง่ายๆ

5. **ความต้านทานการเกิดการชน (Collision Resistance)**

- ไม่ควรสามารถหาสองค่าอินพุตที่ต่างกันแต่ให้ค่าแฮชเหมือนกันได้ในเวลาเชิงสมเหตุสมผล ความสามารถในการป้องกันการชนเป็นสิ่งสำคัญเพื่อรักษาความปลอดภัยของข้อมูล

6. **ผลกระทบแบบ Avalanche (Avalanche Effect)**

- การเปลี่ยนแปลงเล็กน้อยในอินพุต เช่น การเปลี่ยนแปลงเพียงบิตเดียว ควรส่งผลให้ค่าแฮชที่ได้มีการเปลี่ยนแปลงอย่างมากและดูเหมือนไม่มีรูปแบบที่สามารถทำนายได้

7. **ความสุ่มปลอม (Pseudorandomness)**

- ผลลัพธ์ของฟังก์ชันแฮชควรมีลักษณะเหมือนสุ่ม ไม่มีรูปแบบหรือลำดับที่สามารถทำนายได้จากค่าอินพุต

## 8. ขนาดผลลัพธ์คงที่ (Fixed Output Length)

- ฟังก์ชันแฮชควรผลิตค่าแฮชที่มีขนาดคงที่ไม่่ว่าจะเป็นข้อมูลอินพุตที่มีขนาดใดก็ตาม เช่น แฮชขนาด 256 บิตสำหรับทุกอินพุต

## 9. การต่อต้านการขยายความยาว (Resistance to Length Extension Attacks)

- สำหรับฟังก์ชันแฮชบางประเภท ควรมีความต้านทานต่อการโจมตีที่พยายามเพิ่มข้อมูลเข้าไปในอินพุตโดยไม่รู้ค่าแฮชเดิม

### ข้อใดเป็นไปได้ไม่ได้ในโลกแห่งความเป็นจริง

ความต้านทานการเกิดการชน (Collision Resistance)

2. จงเขียนประโยคคำสั่ง (Statement) เพื่อใช้แทนฟังก์ชัน HashEntry ใน member function insert ของ Class HashTable โดยใน member function insert มีการเรียกใช้งานฟังก์ชัน HashEntry ดังนี้  
`array[ currentPos ] = HashEntry( x, ACTIVE );`  
 ตอบ:

```
array[currentPos].element = x;
array[currentPos].info = ACTIVE;
```

3. ในการทำ Quick Sort ขั้นตอนการทำ Partitioning แต่ละรอบนั้น พบว่า มีข้อมูลบางตัวที่ไม่ได้นำไปทำ Partition ด้วยถามว่า มีข้อมูลตัวใดใน array ที่ไม่ได้ถูกทำ Partition เพราะเหตุใด  
 ตอบ: ในการทำ **Quick Sort** ขั้นตอนการทำ **Partitioning** แต่ละครั้งจะมีการเลือก **Pivot** (จุดกึ่งกลาง) เพื่อแบ่งข้อมูลใน array ออกเป็นสองส่วน คือ ส่วนที่มีค่าต่ำกว่า pivot และส่วนที่มีค่าสูงกว่า pivot หลังจากการแบ่งนี้ **Pivot** จะถูกวางไว้ในตำแหน่งที่ถูกต้องใน array ซึ่งหมายความว่า **Pivot** นั้นไม่จำเป็นต้องถูกทำการ partition อีกครั้งในรอบถัดไป

ดังนั้น ข้อมูลที่ไม่ได้ถูกทำ Partition ในแต่ละรอบคือ **Pivot** เอง เนื่องจาก:

1. **Pivot** ได้ถูกจัดวางไว้ในตำแหน่งที่ถูกต้องแล้วใน array
2. ไม่มีความจำเป็นต้องเปรียบเทียบหรือย้าย **Pivot** อีก เนื่องจากมันอยู่ในตำแหน่งที่ถูกต้องตามลำดับ

**สรุป:** ในแต่ละรอบของการทำ Partitioning ใน Quick Sort ข้อมูลที่เป็น **Pivot** จะไม่ได้ถูกนำไปทำ Partition อีก เพราะมันได้ถูกจัดวางไว้ในตำแหน่งที่ถูกต้องเรียบร้อยแล้ว

4. Complete Binary Tree ที่มีความสูงเท่ากับ 16 จะมีจำนวน node อย่างน้อยที่สุดและมากที่สุดเท่าไร ให้ตอบเป็นจำนวนเต็มของจำนวน node โดยไม่ให้อยู่ในรูปของเลขยกกำลัง

ตอบ: ใน Complete Binary Tree ที่มีความสูงเท่ากับ 16:

ใน Complete Binary Tree ที่มีความสูงเท่ากับ 16:

- จำนวนโหนดขั้นต่ำ คือ

$$2^{16} = 65,536 \text{ โหนด}$$

- จำนวนโหนดสูงสุด คือ

$$2^{17} - 1 = 131,071 \text{ โหนด}$$

ดังนั้น:

- จำนวนโหนดขั้นต่ำ: 65,536
- จำนวนโหนดสูงสุด: 131,071



5. กราฟครบ (Complete Graph) ที่มีจำนวนจุดยอด (vertex) เท่ากับ 16 จะมีจำนวนเส้นเชื่อม (edge) ทั้งหมดเท่าไร? จงตอบเป็นจำนวนเต็มของจำนวนเส้นเชื่อม (edge) เท่านั้น โดยไม่ต้องแสดงในรูปของสมการ.

ตอบ: 120

วิธีคำนวณจำนวนเส้นเชื่อม (edge) ในกราฟครบที่มีจำนวนจุดยอด (vertex) เท่ากับ  $n$  คือใช้สูตร:

$$\text{จำนวนเส้นเชื่อม} = \frac{n \times (n - 1)}{2}$$

เหตุผล:

1. การเชื่อมต่อระหว่างจุดยอด: ในกราฟครบ แต่ละจุดยอดจะเชื่อมต่อกับจุดยอดอื่นทั้งหมด ซึ่งมี  $n - 1$  จุดยอดที่เหลือ
  2. รวมจำนวนการเชื่อมต่อ: ดังนั้น จำนวนการเชื่อมต่อทั้งหมดคือ  $n * (n - 1)$
  3. การนับเส้นเชื่อมที่ไม่ซ้ำกัน: เนื่องจากเส้นเชื่อมระหว่างจุดยอด A และ B กับระหว่าง B และ A เป็นเส้นเดียวกัน เราจึงต้องหารด้วย 2 เพื่อไม่ให้นับซ้ำ
- นำมาคำนวณสำหรับ  $n = 16$

$$\begin{aligned} \text{จำนวนเส้นเชื่อม} &= \frac{16 \times (16 - 1)}{2} \\ &= \frac{16 \times 15}{2} \\ &= \frac{240}{2} \\ &= 120 \end{aligned}$$

อื่นๆที่น่าสนใจ

# ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

สรุปสมการและสูตรที่สำคัญ



un	สมการหลักหรือสูตร
Hashing	$\text{hash1}(\text{key}) = \text{key} \bmod \text{table\_size}$ $\text{hash2}(\text{key}) = \text{prime} - (\text{key} \bmod \text{prime})$ $\text{new\_index} = (\text{hash1}(\text{key}) + i \times \text{hash2}(\text{key})) \bmod \text{table\_size}$
Heap/Priority Queue (1-Based Indexing)	$\text{parent}(i) = \lfloor \frac{i}{2} \rfloor$ $\text{left}(i) = 2i$ $\text{right}(i) = 2i + 1$ <b>Heapify Up:</b> เปรียบเทียบและสลับกับพ่อจนกว่าจะไม่เป็น max-heap หรือ min-heap <b>Heapify Down:</b> เปรียบเทียบและสลับกับลูกจนกว่าจะไม่เป็น max-heap หรือ min-heap
Queue	<b>Enqueue (แทรกข้อมูลเข้า):</b> $\text{Rear} = (\text{Rear} + 1) \bmod \text{capacity}$ <b>Dequeue (นำข้อมูลออก):</b> $\text{Front} = (\text{Front} + 1) \bmod \text{capacity}$ <b>ตรวจสอบสถานะ:</b> ว่าง: $\text{Front} = -1$ เต็ม: $(\text{Rear} + 1) \bmod \text{capacity} = \text{Front}$
Stack	<b>Push (แทรกข้อมูลเข้า):</b> $\text{Top} = \text{Top} + 1$ $\text{stack}[\text{Top}] = \text{value}$ <b>Pop (นำข้อมูลออก):</b> $\text{value} = \text{stack}[\text{Top}]$ $\text{Top} = \text{Top} - 1$ <b>ตรวจสอบสถานะ:</b> ว่าง: $\text{Top} = -1$ เต็ม: $\text{Top} = \text{capacity} - 1$
Binary Tree	$\text{parent}(i) = \lfloor \frac{i}{2} \rfloor$ $\text{left}(i) = 2i$ $\text{right}(i) = 2i + 1$ <b>Traversal:</b> Inorder: ซ้าย, ราก, ขวา Preorder: ราก, ซ้าย, ขวา Postorder: ซ้าย, ขวา, ราก <b>ความซับซ้อนเวลา:</b> $O(n)$ สำหรับ Traversal, $O(h)$ สำหรับการค้นหา, แทรก, ลบ (โดยที่ $h$ คือความสูงของต้นไม้)
Sorting	<b>Quick Sort:</b> $\text{Partition}(A, \text{low}, \text{high})$ <b>Merge Sort:</b> $\text{Merge}(L, R)$ <b>Heap Sort:</b> $\text{BuildMaxHeap}(A)$ $\text{Swap}(A[1], A[n])$ $\text{Heapify}(A, n - 1, 1)$ <b>ความซับซ้อนเวลา:</b> Quick Sort: $O(n \log n)$ ในกรณีเฉลี่ย

	Merge Sort: $O(n \log n)$ Heap Sort: $O(n \log n)$ Insertion Sort, Selection Sort, Bubble Sort: $O(n^2)$
Graph	<b>Adjacency List:</b> $\text{List}[v] = \{u_1, u_2, \dots, u_k\}$ <b>Adjacency Matrix:</b> $\text{Matrix}[i][j] = \begin{cases} 1 & \text{ถ้ามีขอบจาก } i \text{ ไป } j \\ 0 & \text{ถ้าไม่มี} \end{cases}$ <b>Traversal:</b> DFS: ใช้ Recursive หรือ Stack BFS: ใช้ Queue
Topological Sort	<b>Kahn's Algorithm:</b> 1. หาโหนดที่ไม่มีขาเข้า (in-degree = 0) 2. ใส่โหนดเหล่านั้นในลำดับ 3. ลบโหนดและขาออกจากกราฟ 4. ทำซ้ำจนกว่าจะไม่มีโหนดเหลือ <b>DFS-based Topological Sort:</b> 1. ทำ DFS บนกราฟ 2. เมื่อสำรวจโหนดเสร็จให้นำโหนดนั้นใส่ใน Stack 3. นำโหนดออกจาก Stack เพื่อสร้างลำดับที่กลับป้อลจิก
Shortest Path	<b>Dijkstra's Algorithm:</b> $d[v] = \min(d[v], d[u] + w(u, v))$ <b>Bellman-Ford Algorithm:</b> $d[v] = \min(d[v], d[u] + w(u, v))$ ทำซ้ำ (

## รายละเอียดเพิ่มเติมสำหรับแต่ละบท

### Hashing

#### Double Hashing:

- ใช้สองฟังก์ชันแฮชเพื่อหาตำแหน่งใหม่เมื่อเกิดการชน
- ช่วยกระจายข้อมูลได้ดีขึ้นและลดการ clustering

#### Rehashing:

- เมื่อตารางแฮชเต็มหรือมีการชนกันมากเกินไป
- ขยายขนาดตารางและคำนวณแฮชใหม่สำหรับทุกคีย์

### Heap/Priority Queue

#### Max-Heap vs Min-Heap:

- Max-Heap:** โหนดแต่ละโหนดมีค่ามากกว่าลูก
- Min-Heap:** โหนดแต่ละโหนดมีค่าน้อยกว่าลูก

#### การใช้งาน:

- Priority Queue ใช้ในอัลกอริทึมเช่น Dijkstra's, A\* Search
- Heap ใช้ในการจัดการข้อมูลแบบเรียงลำดับและการหา k-largest/k-smallest elements

## Queue

ประเภทของ Queue:

- **Circular Queue:** ใช้ modulo เพื่อจัดการตำแหน่ง Front และ Rear
- **Priority Queue:** คิวที่มีการจัดลำดับความสำคัญของแต่ละองค์ประกอบ
- **Deque (Double-Ended Queue):** สามารถแทรกและลบได้ทั้งสองด้าน

## Stack

การใช้งาน:

- การดำเนินการแบบย้อนกลับ (Backtracking)
- การประมวลผลทางภาษา เช่น การตรวจสอบวงเล็บ
- การเรียกใช้งานฟังก์ชัน (Call Stack)

## Binary Tree

ประเภทของ Binary Tree:

- **Binary Search Tree (BST):** แต่ละโหนดมีค่าต่ำกว่าหรือเท่ากับข้างซ้าย และสูงกว่าหรือเท่ากับข้างขวา
- **Balanced Trees (เช่น AVL, Red-Black Trees):** ต้นไม้ที่มีความสูงสมดุลเพื่อเพิ่มประสิทธิภาพการค้นหา
- **Complete Binary Tree:** ทุกระดับเต็ม (ยกเว้นระดับสุดท้าย)
- **Full Binary Tree:** ทุกโหนดมีลูกเต็มสองลูก

การดำเนินการใน Binary Tree:

- **Insertion:** ค้นหาตำแหน่งที่เหมาะสมตามกฎของต้นไม้
- **Deletion:** มีสามกรณีหลัก (โหนดไม่มีลูก, มีลูกเดียว, มีลูกสอง)